

Q - How JS Runs

Ans - JS Runs Top to Bottom

~ It waits for previous line to execute before going to next line

Ex ~ `const pizza = '🍕'` ~ creates just
`console.log(`${pizza}`)` ~ Then this
will execute

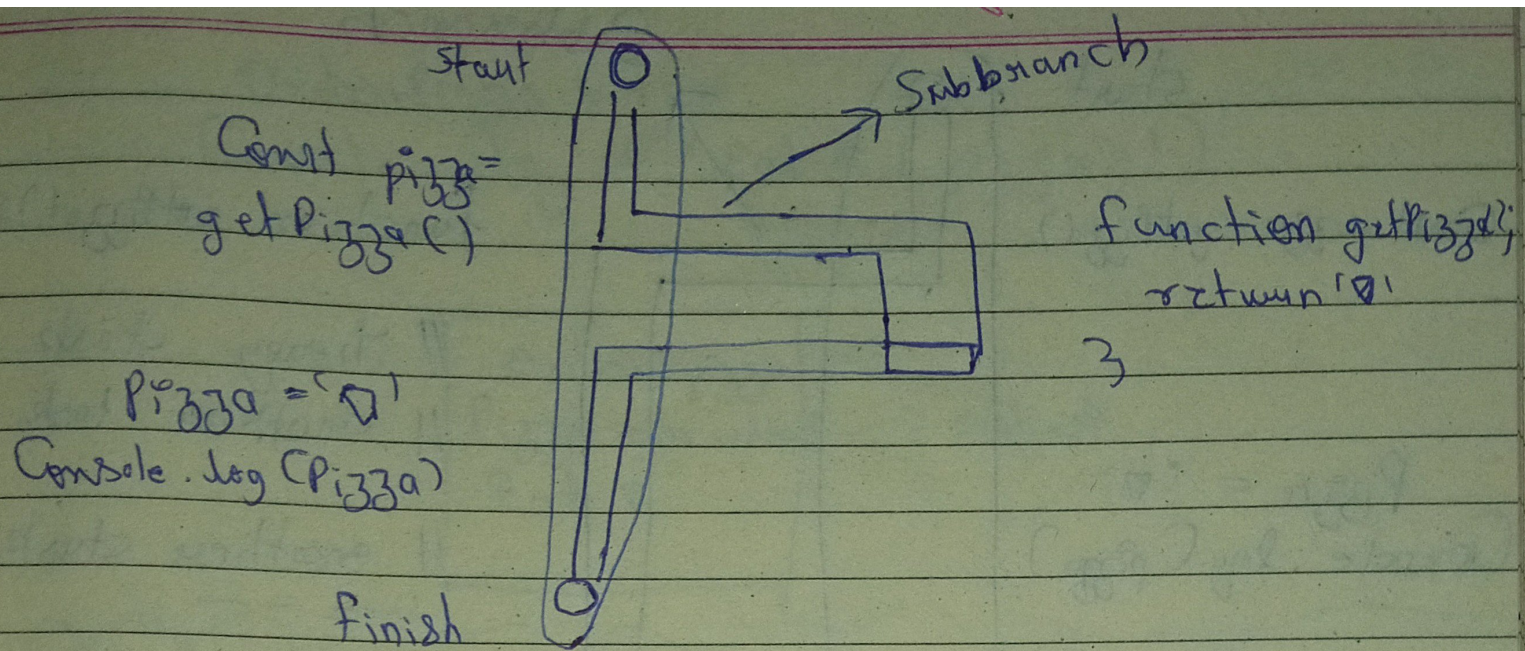
Q - What is Synchronous JS ?

~ In Synchronous JS,

- Each line waits for the previous line to finish.
- Code is executed line by line in the exact order it is written.

Ex ~ `function getPizza() {
 return '🍕'
}`

`const pizza = getPizza()
console.log(`${pizza}`)`



Q- What's the problem with Synchronous JS

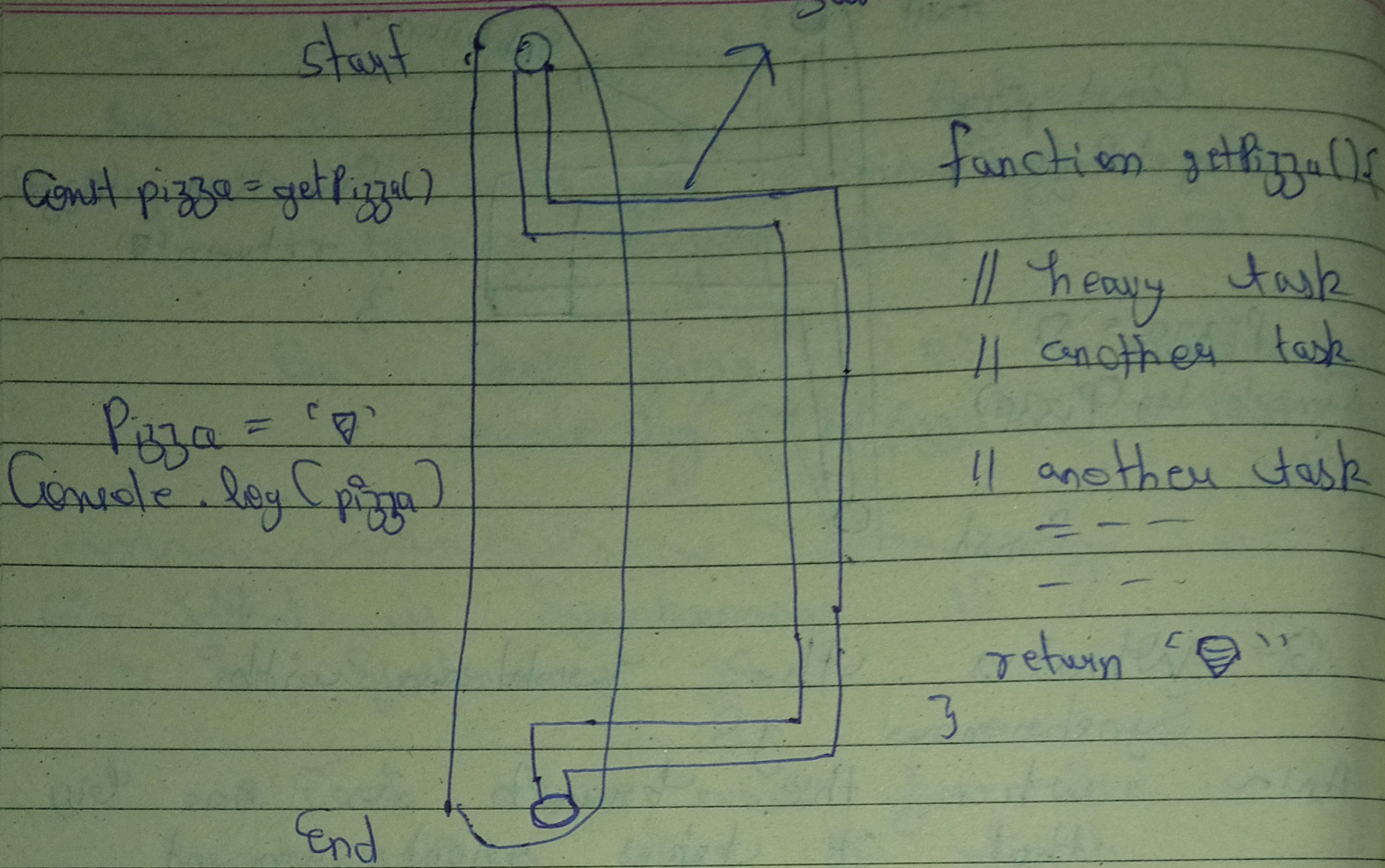
Ans~ when the branch is so big that it takes great amount of time or

When the previous line is taking too much time, getting executed.

Analogy :- Let's say you order pizza, but the delivery guy is taking too much time to deliver it.

So you have to wait for a long time, before you can eat pizza.

Subbranch



* Synchronous code when pizza delivery guy doesn't take time

let pizza

```

function orderPizza() {
  Console.log('order pizza')
  pizza = '🍕'
  Console.log('pizza was ordered')
}
  
```

OrderPizza ()
Console . log ('Eat \$d pizza'))

Order pizza Pizza was ordered Eat ∇	→ output
--	----------

* when pizza delivery guy takes time
let pizza

```
function orderPizza () {
  Console . log
  Set Timeout ( () => {
    pizza = 'D'
  }, 2000 )
}
```

Console . log ('Pizza was ordered')

3

orderPizza ()
Console . log ('Eat \$d pizza')

order pizza pizza was ordered Eat undefined	→ output
---	----------

Q - Why we can't eat pizza this time
or what was the problem, then
eat undefined?

Ans - SetTimeout is asynchronous, so it
schedules the functn to run
after 2 sec

~ So JS doesn't wait for the
pizza to be assigned.

~ It immediately runs
Console.log ("Eat" + pizza)

and since pizza isn't defined yet,
it gives pizza undefined.

Time

what happens

0ms

orderPizza() is called

0ms

Logs order Pizza

0ms

schedules pizza = '🍕' after 2 seconds

0ms

Log pizza was ordered

0ms

immediately logs Eat undefined

2000ms

pizza = '🍕', runs, but it's too
late now.

Asynch JS

~ ~~the~~ Asynch JS allows the program to perform task without blocking the main thread

[i.e. it doesn't wait for a task to complete before moving on to the next]:

Common asynchronous features in JS

- ~ setTimeout(), setInterval(),
- ~ Promises
- ~ Async / Await
- ~ fetch for HTTP requests
- ~ Event listeners
- ~ Callback

Async await

- ~ In order to use await, we need to use Async
- ~ In other words
- ~ Awaits is only usable inside functions that are Async

Q ~ How to make function Async

Ans - just add async keyword before a function ex ~ Async function start() {

Code

```
function getDate () {  
  return new Promise (resolve => {  
    setTimeout ( () => {  
      resolve (46)
```

```
    }, 1)  
  })  
}
```

~

```
async function start () {  
  const result = await getDate ()  
  console.log (result)  
}
```

~

```
start ()
```

46 — output

Fetch

~ When we are getting error from fetch means

There are two types of errors we could come across

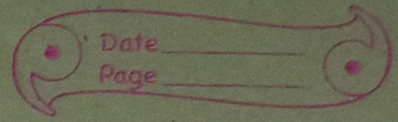
(i) Fetch error ~ your fetch url is wrong or server is unreachable etc

(ii) Server Error
~ fetch operation was successful
~ but the server didn't replied with correct data you wanted.

Fetch handler

~ If there is fetch error, it will still give response instead of an error msg because it considers fetch error only valid type of error
~ Hence we have to write try catch manually in order to get the error msg on server error.

Arrow function



→ Arrow functions can be used in these ways

Basic Syntax (parameters) $\Rightarrow$ { func body }

① No parameters

const sayHello = () $\Rightarrow$ {

console.log("Hello");
}

② Single parameter

const double = (n) $\Rightarrow$ { n * 2 };

const double = n $\Rightarrow$ n * 2;

③ multiple parameters

const multiply = (x, y) $\Rightarrow$ {
 return x * y;
}

console.log(multiply(4, 5));

④ Assigned to a variable

```
const greet = () => {  
  console.log("Hello!");  
};  
  
greet();
```

⑤ Passed directly as argument

```
setTimeout(() => {  
  console.log("Timer done!");  
}, 1000);
```

① ↓
() => {
 console.log("Timer done!");
}

② ↓
1000

setTimeout(1, 2); ~~2000~~

setTimeout(() => console.log("Timer done"), 1000);

Node JS

Used for

- ~ Building APIs
- ~ Server rendered apps
- ~ web servers
- ~ Realtime applications (like chat)
- ~ bot

-
- ~ No window object in node.js
 - ~ No DOM
 - ~ Here we have global variables.

-
- ~ Create a separate file
write a function in it
and export it
 - ~ Now from a diff file import
that file
and now u can use that
function

~ you can use function objects
variables of a separate file into
your current file, if you
imported that file in your
current file.